

Documentazione del Progetto d'Esame

Sistema di Gestione Orario Lezioni

Renato Milo (Matricola: DE1000252)

Giugno 2026

Indice

1	Requisiti del Sistema e Architettura Software	3
1.1	Riassunto dei Requisiti e della Traccia	3
1.2	La Filosofia del Pattern BCE (Boundary-Control-Entity)	3
1.3	Integrazione del Database e Architettura Ibrida BCE + DAO	3
1.3.1	I vantaggi del Pattern BCE + DAO	4
1.3.2	La Gestione dello Stato	4
1.4	Principi di Progettazione dell'Interfaccia Grafica	4
1.4.1	Principio di Leggerezza della GUI	4
1.4.2	Separazione tra i Package	5
1.4.3	Comunicazione e Ciclo di Vita delle Finestre	5
2	Lo Strato delle Entità: Il Package model	6
2.1	Gerarchia degli Utenti e Ereditarietà	6
2.2	Relazioni Strutturali di Calendario e Pianificazione	6
2.3	Tabella Riassuntiva dello Strato delle Entità	7
3	Lo Strato di Boundary: Il Package gui	8
3.1	Schermata di Accesso: LoginFrame	8
3.2	Schermata di Registrazione: RegistrazioneFrame	9
3.3	Pannello dello Studente: DashboardStudente	9
3.4	Pannello del Docente: DashboardDocente	10
3.5	Pannello del Coordinatore: DashboardResponsabile	10
3.6	Pianificazione delle Attività: InserimentoLezioneFrame	11
4	Lo Strato di Coordinamento e Controllo: Il package controller	14
4.1	Introduzione Teorica: Il ruolo del Control nel Pattern BCE + DAO	14
4.1.1	Gestione della Sessione Utente e Menu Dinamici nella GUI	14
4.2	Analisi dei Campi di Classe e Costruttore	15
4.3	Analisi Operativa dei Metodi di Inizializzazione e Sincronizzazione	15
4.3.1	Il Metodo getInstance()	15
4.3.2	Il Metodo sincronizzaConDatabase()	16
4.3.3	I Metodi di Autenticazione e Sessione	16
4.3.4	I Metodi di Scrittura Transazionale (PostgreSQL + Caching)	17
4.3.5	I Metodi di Consultazione della Cache (Query in Memoria)	18
4.4	Tabella Riassuntiva dello Strato di Controllo	18

5	Lo Strato di Persistenza dei Dati: Il Pattern DAO	20
5.1	La Gestione della Connessione: Il Package database_connection	20
5.1.1	Gestione delle Risorse JDBC e Sicurezza nelle Query	20
5.2	Le Interfacce d'Accesso: Il Package dao	20
5.3	L'Implementazione PostgreSQL: Il Package implementazioneDao	21
5.4	Tabella Riassuntiva delle Operazioni del DAO	21
5.5	Dettaglio dell'Implementazione Fisica dei Data Access Objects	23
5.5.1	La classe UtenteImplementazionePostgresDAO	23
5.5.2	La classe AulaImplementazionePostgresDAO	24
5.5.3	La classe InsegnamentoImplementazionePostgresDAO	24
5.5.4	La classe LezioneImplementazionePostgresDAO	24
5.5.5	La classe RichiestaSpostamentoImplementazioneDAO	24
6	Sull'utilizzo delle Mappe	24
6.1	Cosa sono le mappe	25
6.2	Uso delle mappe	25
6.2.1	Dai Cicli Nidificati all'uso delle HashMap	25

1 Requisiti del Sistema e Architettura Software

1.1 Riassunto dei Requisiti e della Traccia

La traccia del progetto prevede lo sviluppo di un sistema informativo per la pianificazione e la gestione dell'orario settimanale delle lezioni di un corso di laurea. Sulla base dei requisiti frontiti nell'ambito della traccia, ho identificato i seguenti obiettivi funzionali primari che il sistema deve necessariamente supportare:

- **Gestione degli Utenti e Abilitazione degli Account:** Il sistema deve consentire la registrazione dei nuovi utenti indicandone anagrafica e credenziali. Gli utenti si suddividono in Studenti e Docenti. Tra i docenti, un utente o diversi assumono il ruolo di Responsabile degli orari e della gestione della didattica del corso (Coordinatore).
- **Definizione delle Risorse Didattiche:** colui che ha il compito del responsabile ha l'onere di andare a definire l'elenco degli insegnamenti attivi (andandone ad indicare nome e CFU), inserire le aule disponibili e creare le lezioni nel calendario settimanale, associandovi un corso, un giorno della settimana, una fascia oraria e un'aula.
- **Workflow di Riprogrammazione delle Lezioni:** Ciascun docente deve poter consultare il proprio calendario didattico personale e, qualora si presentasse la necessità, inoltrare una richiesta formale di spostamento di una lezione, indicando la fascia oraria originaria e quella proposta potendo. Il Responsabile visualizza queste richieste in sospeso e può decidere se approvarle (aggiornando l'orario a sistema) o rifiutarle.
- **Consultazione degli Orari:** Gli studenti devono poter visualizzare l'orario completo delle lezioni relativo al proprio anno di corso, con l'indicazione esatta delle aule assegnate.

1.2 La Filosofia del Pattern BCE (Boundary-Control-Entity)

Essendo un requisito potere mantenere un'elevata manutenibilità, estensibilità e una chiara separazione delle responsabilità, è stato seguito, come concordato col docente, il pattern di realizzazione BCE:

- **Boundary (Package gui):** Rappresenta il livello di interazione e confine del sistema con l'esterno. Nel progetto realizzato, coincide con le interfacce grafiche sviluppate in Java Swing. Questo strato si occupa esclusivamente di acquisire gli input dell'utente, effettuare validazioni formali elementari (es. campi vuoti) e mostrare i dati in uscita, seguendo, dunque il principio di leggerezza della gui.
- **Control (Package controller):** Rappresenta lo strato di coordinamento e contenente la logica dell'applicazione. È centralizzato in un'unica classe `Controller.java` (attendendo alle indicazioni concordate) che mediando tra l'interfaccia di presentazione e i dati di dominio garantisce la separazione degli ambiti di lavoro.
- **Entity (Package model):** Rappresentante del modello informativo e contiene le classi che riproducono le informazioni del dominio reale (Utenti, Aule, Lezioni, Insegnamenti, Richieste) mappate tramite oggetti Java.

1.3 Integrazione del Database e Architettura Ibrida BCE + DAO

Nell'architettura BCE pura, i dati sono gestiti esclusivamente in memoria volatile dalle classi del *Model*, comportando la perdita totale delle informazioni alla chiusura dell'applicazione, volendo ottemperare alla traccia è stato esteso al modello BCE+DAO.

1.3.1 I vantaggi del Pattern BCE + DAO

Il sopracitato pattern separa la descrizione dei requisiti di persistenza (le firme dei metodi di accesso, definite in una serie di interfacce nel package `dao`) dalla loro implementazione fisica (le query SQL connesse a PostgreSQL nel package `implementazioneDao`).

1.3.2 La Gestione dello Stato

Un punto importante dell'architettura del software riguarda il modo in cui vengono gestiti i dati tra la memoria volatile dell'applicazione e il database vero e proprio. Ho cercato di separare nettamente queste due parti sia per ottimizzare le prestazioni sul client, sia per non appesantire il database con continue richieste.

- **Dati Transienti (Package model e controller):** In linea con il pattern BCE, le classi del model mantengono i dati in memoria solo finché l'applicazione è attiva. Il **Controller** memorizza gli oggetti principali in liste dedicate (`List<Utente>`, `List<Lezione>`, ecc.), che funzionano come una specie di cache locale. In questo modo, quando l'utente deve semplicemente consultare o filtrare i dati (ad esempio per vedere le lezioni di uno studente), l'operazione avviene direttamente in RAM alla massima velocità, evitando di inviare continuamente query `SELECT` verso il database.
- **Dati Persistenti (Package dao e implementazioneDao):** Per salvare i dati a lungo termine, oltre la chiusura del programma, ci si affida al database PostgreSQL. In questo strato i dati perdono la loro struttura a oggetti per essere organizzati in tabelle relazionali, collegate tra loro tramite vincoli di integrità e chiavi esterne (*Foreign Key*).

Per collegare la struttura a oggetti di Java con quella tabellare del database, il **Controller** coordina l'allineamento dei dati usando due strategie diverse a seconda dell'operazione:

1. **Aggiornamento Diretto :** Per le modifiche più semplici e immediate (come l'approvazione o il rifiuto di una richiesta), il **Controller** chiama il DAO per aggiornare il database. Se la transazione va a buon fine, modifica subito anche lo stato dell'oggetto già presente in memoria, mantenendo l'interfaccia aggiornata senza dover ricaricare tutto il dataset.
2. **Sincronizzazione Completa (sincronizzaConDatabase):** Per operazioni più complesse o all'avvio delle schermate, il sistema adotta un approccio dall'alto verso il basso. Il metodo svuota completamente le liste locali in RAM e rilegge i dati dal database tramite i DAO. Sfruttando delle Map d'appoggio basate sugli ID, il controller ricostruisce da zero i collegamenti tra gli oggetti (ad esempio riassegnando ogni lezione alla sua aula), evitando così il rischio di lasciare riferimenti orfani o dati disallineati.

1.4 Principi di Progettazione dell'Interfaccia Grafica

Nello sviluppo delle interfacce grafiche con la tecnologia Swing UI Designer di IntelliJ, ho seguito tre principi guida essenziali per garantire la robustezza e la pulizia del codice:

Lo sviluppo dell'interfaccia utente con la tecnologia Swing è stato realizzato tenendo in mente i seguenti principi architetturali, volti alla manutenibilità e alla corretta gestione delle risorse:

1.4.1 Principio di Leggerezza della GUI

Le classi del pacchetto `gui` contengono il minimo di codice possibile. Tutte le elaborazioni complesse dei dati, i calcoli e il controllo dei vincoli logici sono interamente demandati alla classe di controllo **Controller**. La GUI si limita a leggere i dati immessi dai widget e a invocare i metodi esposti dal controllore. Cioè ottemperando ed adempiendo ai principi sanciti dal pattern architetturale che viene implementato.

1.4.2 Separazione tra i Package

Le classi grafiche sono inserite in modo esclusivo nel package `gui`, ergo isolandole completamente dal resto del software . Questa netta separazione consente altresì di poter modificare lo stile della GUI, alterare i layout senza dover toccare la logica del Controller o la struttura del database, ciononostante la gestione del comparto visivo del programma è stato realizzato mantenendo la coerenza grafica (sebbene le mie capacità di immaginare un comparto grafico accattivante lascino sicuramente a desiderare).

1.4.3 Comunicazione e Ciclo di Vita delle Finestre

Nel progetto è stato evitato il passaggio diretto di dati tra le diverse finestre grafiche . La comunicazione riguarda solo il passaggio del controllo: il frame chiamante passa il riferimento a se stesso ed al costruttore del frame chiamato e si rende invisibile. Al termine delle operazioni, il frame chiamato riattiva il chiamante ed esegue il metodo `dispose()` per distruggersi fisicamente, liberando le risorse e permettendo la corretta deallocazione della memoria da parte del Garbage Collector della Jvm

2 Lo Strato delle Entità: Il Package model

Lo strato delle entità, implementato nel package `model`, rappresenta la rappresentazione in oggetti Java delle informazioni concettuali ricavate dalla traccia e rappresentata nel class diagram.

Di seguito vengono riportate le entità le quali sono state inserite per realizzare il progetto

2.1 Gerarchia degli Utenti e Ereditarietà

Usufruendo del principio di ereditarietà ho realizzato le classi relative a coloro i quali vanno ad agire sul sistema:

- **Utente (Classe Astratta):** Raccoglie gli attributi anagrafici (*nome*, *cognome*, *email*) e di autenticazione (*login*, *password*). Ho dichiarato questa classe come **abstract** poiché non ha senso istanziare un utente generico senza un ruolo specifico.
- **Studente (Sottoclasse di Utente):** Specializza la classe padre introducendo la *matricola* e l'*anno* di corso (I, II o III). Ho deciso che la matricola non venisse digitata a mano, ma generata in automatico dal Database Management System PostgreSQL con la sequenza d'ateneo (DE/1) per semplificare il form di registrazione.
- **Docente (Sottoclasse di Utente):** Rappresenta il corpo docente. Possiede una lista di insegnamenti di cui è titolare, implementando un'associazione "uno-a-molti" con la classe *Insegnamento* per consentire una facile navigazione tra docenti e corsi.
- **Responsabile (Sottoclasse di Docente):** Identifica il docente con poteri di coordinatore degli orari, non introducendo nuovi attributi nel modello, eredita tutte le proprietà del docente, distinguendosi però per i casi d'uso che è abilitato a richiamare nello strato di controllo.

2.2 Relazioni Strutturali di Calendario e Pianificazione

Le restanti classi del modello governano la pianificazione settimanale e le aule:

- **Insegnamento:** Rappresenta un corso del piano di studi (caratterizzato da CFU e anno). Mantiene una relazione di composizione forte con le proprie lezioni. La scelta della composizione è stata effettuata in quanto a livello logico una lezione non può esistere in calendario se non è legata a un insegnamento di riferimento; l'eventuale rimozione del corso comporta la distruzione a cascata di tutte le sue lezioni associate.
- **Lezione:** È l'unità fondamentale del calendario. Contiene i riferimenti all'insegnamento di appartenenza, all'aula fisica assegnata e alla finestra oraria (*ora_inizio*, *ora_fine*). Ho inserito in questa classe dei metodi *setter* per consentire la riprogrammazione di giorno e orario a seguito dell'approvazione di una richiesta.
- **Aula:** Rappresenta lo spazio fisico dell'ateneo. Mantiene una relazione di aggregazione con le lezioni ospitate: un'aula esiste a prescindere dalle lezioni pianificate in essa. Ho inserito in questa entità il metodo critico `isLibera` per verificare, mediante un ciclo di controllo temporale, che non avvengano sovrapposizioni fisiche di lezioni nella stessa aula.
- **RichiestaSpostamento:** Modella la richiesta inviata da un docente per variare l'orario di una lezione. Contiene il riferimento alla lezione originaria e i nuovi dettagli proposti, oltre allo stato della richiesta (in attesa, approvata, rifiutata).

2.3 Tabella Riassuntiva dello Strato delle Entità

Per fornire un quadro sintetico dell'implementazione dello strato delle entità, ho riassunto nella Tabella 1 l'importanza di ciascuna classe nel sistema e l'elenco completo dei suoi metodi Java reali.

Entità	Importanza nel Sistema	Elenco dei Metodi Java
Utente	Fornisce la struttura di base e le credenziali per l'accesso e l'anagrafica di tutti gli attori.	Costruttore, <code>getNome()</code> , <code>getCognome()</code> , <code>getLogin()</code> , <code>getPassword()</code> , <code>getIdUtente()</code> , <code>setIdUtente()</code>
Studente	Consente l'accesso personalizzato agli studenti e filtra la visualizzazione dell'orario in base al proprio anno.	Costruttore, <code>getMatricola()</code> , <code>getAnno()</code>
Docente	Rappresenta l'insegnante abilitato a visionare il proprio calendario e a proporre modifiche d'orario.	Costruttore, <code>addInsegnamento()</code> , <code>getInsegnamentiTitolare()</code> , <code>toString()</code>
Responsabile	Identifica l'amministratore del sistema con i privilegi di pianificazione ed approvazione.	Costruttore
Insegnamento	Rappresenta il corso accademico, legando CFU e anno di corso al docente titolare.	Costruttore, <code>addLezione()</code> , <code>getNome()</code> , <code>getDocenteTitolare()</code> , <code>getAnnoDiCorso()</code> , <code>getIdInsegnamento()</code> , <code>setIdInsegnamento()</code> , <code>toString()</code>
Lezione	Definisce l'unità di orario schedulata, con i riferimenti all'insegnamento e all'aula assegnati.	Costruttore, <code>getGiorno()</code> , <code>getOraInizio()</code> , <code>getOraFine()</code> , <code>getAula()</code> , <code>getInsegnamento()</code> , <code>getIdLezione()</code> , <code>setIdLezione()</code> , <code>setGiorno()</code> , <code>setOraInizio()</code> , <code>setOraFine()</code>
Aula	Rappresenta l'aula fisica e si fa carico di controllare l'assenza di conflitti e sovrapposizioni orarie.	Costruttore, <code>addLezione()</code> , <code>isLibera()</code> , <code>getNome()</code> , <code>getIdAula()</code> , <code>setIdAula()</code> , <code>toString()</code>
Richiesta	Incapsula i dati della proposta di riprogrammazione di una lezione e ne traccia lo stato.	Costruttore, <code>getLezioneDaSpostare()</code> , <code>getGiornoProposto()</code> , <code>getOraInizioProposta()</code> , <code>getOraFineProposta()</code> , <code>getStato()</code> , <code>setStato()</code> , <code>getIdRichiesta()</code> , <code>setIdRichiesta()</code>

Tabella 1: Dizionario di dettaglio dello strato delle Entità (Model).

3 Lo Strato di Boundary: Il Package gui

Lo strato di Boundary, implementato interamente all'interno del package `gui`, costituisce l'interfaccia di presentazione visiva dell'applicazione ed è stato interamente sviluppato avvalendosi della libreria standard **Java Swing** integrata mediante lo strumento visuale *Swing UI Designer* di IntelliJ.

In questa fase finale del progetto, ho ampliato notevolmente lo strato di presentazione per supportare tutte le operazioni di persistenza, strutturando la navigazione tra le finestre in modo che rispetti rigorosamente i vincoli di disaccoppiamento ed efficienza di memoria imposti in sede di analisi :

1. **Gestione della memoria tramite `dispose()`:** Volendo ottimizzare lo spazio occupato sulla Java Virtual Machine (JVM), è stato evitato l'uso di un unico `JFrame` in cui i pannelli vengono mostrati e nascosti all'occorrenza. Al contrario, ho deciso di istanziare frame indipendenti a disegno statico: quando un frame secondario viene chiuso per tornare alla schermata precedente, esso invoca il metodo `dispose()`, distruggendo fisicamente la finestra e rendendo l'oggetto immediatamente deallocabile da parte del Garbage Collector.
2. **Accesso Centralizzato tramite Pattern Singleton:** Al fine di garantire un unico punto di accesso coerente e centralizzato allo stato dell'applicazione, si è evitato il passaggio manuale del riferimento al **Controller** attraverso i costruttori delle diverse finestre di Boundary. Al contrario, si è implementato il pattern creazionale *Singleton* sulla classe `Controller.java`. Ciascun frame grafico (come `LoginFrame`, `DashboardDocente`, ecc.) interroga in modo indipendente l'unica istanza attiva del controllore mediante l'invocazione statica del metodo `Controller.getInstance()`.

Di seguito si riportano nel dettaglio le singole finestre che compongono l'interfaccia utente dell'applicazione, corredate dai relativi screenshot di esecuzione.

3.1 Schermata di Accesso: `LoginFrame`

La classe `LoginFrame` rappresenta l'entry point grafico dell'applicazione. Consente l'autenticazione degli utenti mediante l'acquisizione delle credenziali di login e password, che vengono inoltrate al *Controller* per la validazione sul database PostgreSQL.

Rispetto alle fasi precedenti, ho integrato in questa interfaccia un pulsante aggiuntivo per consentire ai nuovi utenti di accedere direttamente alla mascherina di registrazione autonoma. Se l'autenticazione va a buon fine, il frame rileva il ruolo dell'utente autenticato, istanziando la corretta dashboard di riferimento

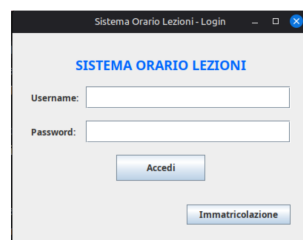


Figura 1: Schermata di autenticazione dell'applicazione (`LoginFrame`).

3.2 Schermata di Registrazione: `RegistrazioneFrame`

Viene sviluppata la classe `RegistrazioneFrame` per consentire l’inserimento e la memorizzazione a database di nuovi utenti del sistema.

La finestra riceve nel costruttore il riferimento del frame chiamante (`LoginFrame` o `DashboardResponsabile`) si rende visibile nascondendo temporaneamente la schermata di provenienza e gestisce l’abilitazione dinamica dei campi : se l’utente seleziona il ruolo "STUDENTE", il sistema abilita l’inserimento dell’anno di corso, mentre omette l’inserimento della matricola, in quanto ho delegato la generazione incrementale di quest’ultima direttamente al database PostgreSQL. Al salvataggio, richiama la persistenza tramite il *Controller*, mostra un messaggio di conferma, riattivando il frame chiamante e viene distrutto

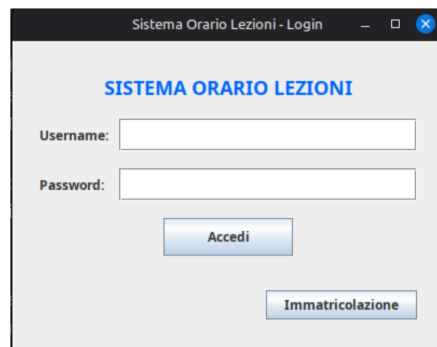


Figura 2: Interfaccia di Login.

3.3 Pannello dello Studente: `DashboardStudente`

Questa schermata costituisce l’area di sola consultazione riservata agli studenti autenticati. All’atto dell’apertura, l’interfaccia interroga il *Controller* per estrarre dal modello in memoria l’orario completo delle lezioni.

I dati vengono mostrati in un componente `JTable` (inserito all’interno di un `JScrollPane` permettendo lo scorrimento dei record), reso esplicitamente non modificabile a runtime per ragioni di sicurezza. Le lezioni visualizzate sono filtrate a monte dal Controller per mostrare unicamente quelle relative allo specifico anno di corso frequentato dallo studente loggato.

Benvenuto, Mario Rossi (DE10000)

Giorno	Ora Inizio	Ora Fine	Materia	Aula
LUNEDI	10:00	12:00	Programmazione O...	Aula A1
GIOVEDI	14:00	16:00	Programmazione O...	Aula A1

Esci (Logout)

Figura 3: Visualizzazione oraria personalizzata per lo studente loggato.

3.4 Pannello del Docente: DashboardDocente

La classe `DashboardDocente` implementa l'interfaccia riservata ai docenti. Mostra in una tabella l'orario delle sole lezioni di cui il professore autenticato

In basso, viene implementato un form integrato per la richiesta di riprogrammazione di una lezione: il docente seleziona la lezione d'interesse cliccando direttamente sulla riga della tabella, sceglie il nuovo giorno proposto tramite un menu a tendina (`JComboBox`) e inserisce i nuovi orari proposti all'interno di campi di testo dedicati. Al clic su "Invia Richiesta", l'interfaccia valida la presenza dei dati e li inoltra al Controller affinché vengano scritti su PostgreSQL.

Cruscotto Docente

Prof. Ignazio Abate

Giorno	Ora Inizio	Ora Fine	Materia	Aula
LUNEDI	12:30	13:00	Stabia	Aula A6

Invia Richiesta di Spostamento Lezione

Giorno: Inizio: Fine: **Invia Richiesta**

Esci

Figura 4: Interfaccia docente per la consultazione e l'invio delle richieste.

3.5 Pannello del Coordinatore: DashboardResponsabile

Questa complessa dashboard rappresenta il centro di amministrazione del sistema, accessibile solo agli utenti registrati come responsabili degli orari.

L'interfaccia mostra l'elenco di tutte le richieste di spostamento d'orario attualmente pendenti sul database. Attraverso due pulsanti interattivi ("Approva Spostamento" e "Rifiuta Richiesta"), il responsabile può decidere l'esito del ticket. In basso, ho inserito i pulsanti di collegamento che permettono al coordinatore di aprire le finestre secondarie per la registrazione di nuovi utenti, la pianificazione di lezioni o l'abilitazione degli account in sospeso.

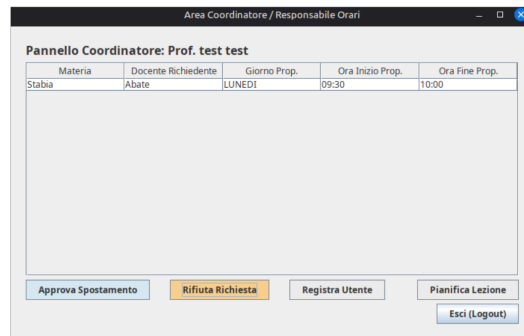


Figura 5: Pannello amministrativo del Responsabile degli orari.

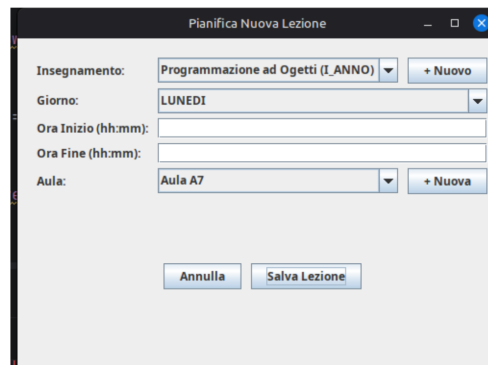


Figura 6: Pannello amministrativo del Responsabile degli orari.

3.6 Pianificazione delle Attività: InserimentoLezioneFrame

Vine progettata la classe `InserimentoLezioneFrame` per consentire al responsabile di creare e inserire nuove lezioni in calendario.

Per massimizzare la facilità d'uso e la flessibilità del posizionamento (evitando di costringere il responsabile a chiudere la finestra se si accorge che un'aula o un insegnamento non sono presenti a sistema), ho implementato due pulsanti contrassegnati con "+" :

- Cliccando su ”+ **Nuovo**” accanto agli insegnamenti, si apre una finestra modale di dialogo (`JOptionPane.showInputDialog`) che permette di inserire sul database un intero nuovo insegnamento (CFU, anno e docente titolare) senza uscire dalla schermata corrente.
- In modo speculare, cliccando su ”+ **Nuova**” accanto alle aule, è possibile inserire all’istante una nuova aula fisica su PostgreSQL.

All’inserimento, i menu a tendina della pianificazione si aggiornano dinamicamente caricando i nuovi dati appena scritti.

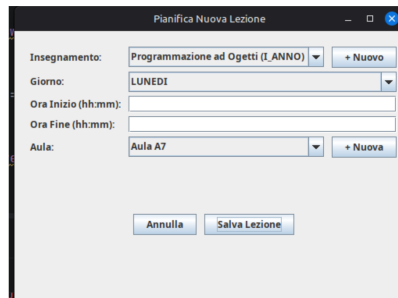


Figura 7: Interfaccia per la pianificazione e l’inserimento dinamico di nuove lezioni.

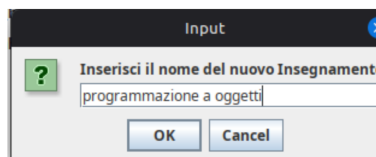


Figura 8: Interfaccia per la pianificazione e l’inserimento dinamico di nuove lezioni.

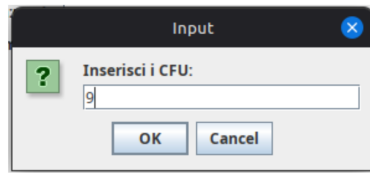


Figura 9: Interfaccia per la pianificazione e l'inserimento dinamico di nuove lezioni.

4 Lo Strato di Coordinamento e Controllo: Il package controller

4.1 Introduzione Teorica: Il ruolo del Control nel Pattern BCE + DAO

All'interno del pattern architetturale **Boundary-Control-Entity (BCE)**, lo strato di **Control** (Controllo) si occupa del coordinamento, fungendo da cerniera e mediando tra l'interfaccia di presentazione con l'esterno (*Boundary*) e il modello informativo che rappresenta i dati reali (*Entity*).

Le responsabilità fondamentali che ho assegnato a questo strato sono le seguenti:

- **Disaccoppiamento e Isolamento:** Le interfacce grafiche non conoscono le classi del modello di dominio, né tantomeno hanno visibilità delle query (cosa che potrebbe creare gravi carenze a livello di sicurezza). Qualsiasi azione scaturita dal click di un bottone sulla GUI viene tradotta in una chiamata a un metodo del *Controller*, che si fa carico di elaborare la richiesta
- **Gestione del Caching Transiente (In-Memory):** Per prevenire la pesantezza prestazionale intrinseca nelle continue interrogazioni di rete verso PostgreSQL, il *Controller* mantiene in memoria delle collezioni temporanee delle entità del sistema, garantendo, dunque una adeguata velocità d'esecuzione dell'applicativo.
- **Coordinamento della Persistenza (In-Memory + DB):** Quando si verifica un'operazione di scrittura o aggiornamento, il *Controller* coordina una doppia transazione: invoca l'interfaccia DAO corrispondente per salvare fisicamente il dato all'interno di PostgreSQL (persistenza a lungo termine) e contemporaneamente aggiorna la propria cache interna in memoria

Al fine di garantire l'unicità di questo "cervello coordinatore", si è deciso di implementare la classe `Controller.java` applicando il pattern creazionale Singleton, garantendo che esista una sola istanza attiva del Controller per tutta la durata dell'esecuzione dell'applicazione, assicurando che lo stato in memoria rimanga sempre coerente e centralizzato, indipendentemente da quale interfaccia grafica stia interrogando il sistema di database.

4.1.1 Gestione della Sessione Utente e Menu Dinamici nella GUI

Per gestire l'utente che ha effettuato l'accesso nel sistema, ho sfruttato i principi dell'orientamento agli oggetti e in particolare il polimorfismo. All'interno del *Controller*, l'attributo che tiene traccia della sessione attiva è dichiarato usando il tipo della classe base astratta: `private Utente utenteLoggato;`

Grazie al meccanismo dell'*Upcasting*, questa singola variabile può contenere a runtime una qualunque istanza delle tre classi figlie: `Studente`, `Docente` o `Responsabile`. Questa scelta offre due vantaggi pratici:

1. **Disaccoppiamento del Codice:** Il *Controller* non deve implementare controlli continui o logiche separate per capire chi è l'utente quando deve eseguire funzioni base (come il logout o la lettura dei dati anagrafici), riducendo l'accoppiamento tra le classi.
2. **Adattamento dei Menu della GUI:** Quando viene caricata la schermata principale, la GUI interroga il controller per ottenere l'utente loggato. Tramite l'uso dell'operatore `instanceof` e di un successivo *downcasting* sicuro, l'interfaccia grafica riconosce il ruolo effettivo dell'utente. In questo modo i menu vengono abilitati dinamicamente: ad esempio, la sezione per valutare e approvare le richieste di spostamento delle lezioni viene mostrata e resa cliccabile soltanto se l'utente è un `Responsabile`, bloccando l'accesso agli altri ruoli già a livello visivo.

4.2 Analisi dei Campi di Classe e Costruttore

La classe `Controller.java` dichiara come attributi privati le liste di cache del finto database in memoria:

```
private List<Utente> utenti;  
private List<Aula> aule;  
private List<Insegnamento> insegnamenti;  
private List<Lezione> lezioni;  
private List<RichiestaSpostamento> richieste;  
private List<Docente> docenti;
```

Inoltre, dichiara come attributi privati i riferimenti astratti alle interfacce DAO del package `dao`:

```
private UtenteDAO utenteDAO;  
private AulaDAO aulaDAO;  
private InsegnamentoDAO insegnamentoDAO;  
private LezioneDAO lezioneDAO;  
private RichiestaSpostamentoDAO richiestaSpostamentoDAO;
```

Il costruttore `private Controller()` è volutamente inaccessibile dall'esterno, al fine di garantire la sicurezza dell'applicazione e il corretto adempimento dei pattern di buone pratiche. Al suo interno si provvede a:

1. Istanziare le collezioni concrete come oggetti di tipo `ArrayList`.
2. Istanziare i connettori fisici del database richiamando le implementazioni concrete del package `implementazioneDao` (ad esempio, `new UtenteImplementazionePostgresDAO()`).
3. Invocare il metodo privato `sincronizzaConDatabase()` per allineare immediatamente lo stato in memoria a quello del database al momento dell'avvio dell'applicazione.

4.3 Analisi Operativa dei Metodi di Inizializzazione e Sincronizzazione

4.3.1 Il Metodo `getInstance()`

Il metodo di accesso globale `public static Controller getInstance()` implementa la inizializzazione del Singleton (unica istanza di accesso al database al fine di garantire l'integrità dello stesso):

```
public static Controller getInstance() {  
    if (instance == null) {  
        instance = new Controller();  
    }  
    return instance;  
}
```

Se richiamato per la prima volta, esegue il costruttore privato allocando la risorsa; per tutte le chiamate successive da parte dei `JFrame`, restituisce semplicemente il riferimento all'oggetto già esistente in memoria.

4.3.2 Il Metodo `sincronizzaConDatabase()`

Questo metodo privato rappresenta l'algoritmo di allineamento iniziale ed è il cuore logico dell'integrazione con PostgreSQL. Poiché le tabelle relazionali del database sono strutture piatte e separate, il metodo deve farsi carico di estrarre le chiavi esterne (FK) e ricostruire graficamente la complessa rete di puntatori e riferimenti ad oggetti del modello ad oggetti.

L'algoritmo opera secondo una sequenza logica di dipendenze ben definita:

1. **Svuotamento dei Dati Preesistenti:** All'avvio o in caso di ricaricamento, invoco il metodo `clear()` su tutte le liste in memoria per evitare ridondanze e duplicazioni orfane.
2. **Caricamento dei Docenti (Tabelle collegate):** Invoco il metodo `leggiTuttiIDocentiIDB` dell'interfaccia `UtenteDAO`. Ricevo i dati grezzi su delle liste di appoggio (per non violare il disaccoppiamento con il Model). All'interno di un ciclo, istanzio gli oggetti del Model polimorfici corretti: se l'attributo `isResponsabile` è vero, creo un oggetto di tipo `Responsabile`, altrimenti un semplice `Docente`. Memorizzo gli oggetti creati in una mappa temporanea `Map<Integer, Docente>` dove la chiave è l'ID reale del database; questa mappa sarà l'ancora referenziale per i passaggi successivi.
3. **Caricamento delle Aule:** Interrogo il database tramite `aulaDAO.leggiAuleDB` e popolo la lista `aule` del Model. Anche in questo caso, memorizzo le istanze create in una mappa d'appoggio `Map<Integer, Aula>` indicizzata con la chiave primaria di PostgreSQL.
4. **Caricamento degli Insegnamenti:** Invoco il metodo `leggiInsegnamentiIDB`. Durante il ciclo di lettura, per ogni riga estratta prelevo l'ID del docente titolare (FK.Docente), interrogo la mappa dei docenti creata al punto 1 (`mapDocenti.get(fkDocente)`) e istanzio l'oggetto `Insegnamento` passando il riferimento dell'oggetto docente reale appena recuperato. Inserisco l'oggetto nella mappa temporanea `Map<Integer, Insegnamento>`.
5. **Caricamento delle Lezioni (La composizione):** Interrogo il database tramite `lezioneDAO.leggiLezioniIDB`. Per ogni record estratto, prelevo la chiave esterna dell'insegnamento e dell'aula. Recupero i rispettivi oggetti reali dalle mappe dei punti precedenti e istanzio l'oggetto `Lezione` passando gli orari convertiti a runtime in oggetti di tipo `LocalTime`. Il costruttore di `Lezione` provvederà autonomamente a registrarsi all'interno delle liste interne di `Insegnamento` e `Aula`, ricomponendo la relazione bidirezionale e di composizione forte. Memorizzo le lezioni in una mappa temporanea `Map<Integer, Lezione>`.
6. **Caricamento delle Richieste di Spostamento:** Infine, interrogo la persistenza tramite `leggiRichiesteDB`. Per ogni richiesta, recupero l'oggetto reale `Lezione` a cui fa riferimento dalla mappa del punto precedente ed istanzio l'oggetto `RichiestaSpostamento`, impostandone lo stato di avanzamento corrente ricavato dal DB (in attesa, approvata o rifiutata).

4.3.3 I Metodi di Autenticazione e Sessione

- **public boolean effettuaLogin(String login, String password):**

Questo metodo gestisce la fase di convalida delle credenziali e l'inizializzazione della sessione utente.

1. Invoca il metodo `loginDB` dell'interfaccia `UtenteDAO`, passando le credenziali digitate sulla GUI. Il DAO esegue una query protetta su PostgreSQL e restituisce una stringa che identifica il ruolo dell'utente ('STUDENTE', 'DOCENTE', 'RESPONSABILE'), oppure `null` in caso di credenziali errate o account non abilitato.

2. Se il ruolo è valido, istanzia sette liste di appoggio vuote e chiama il metodo `leggiDatiUtenteLoggato` per estrarre in modo sicuro i dati anagrafici dal database relazionale, aggirando qualsiasi dipendenza del DAO dagli oggetti del modello.
3. Sulla base del ruolo restituito dal DB, istanzia a runtime l'oggetto del Model corretto (`Studente`, `Docente` o `Responsabile`) richiamandone il rispettivo costruttore, assegna l'ID utente recuperato dal database e lo memorizza nell'attributo privato `utenteLoggato`.
4. Infine, restituisce `true` per indicare alla GUI del Login che l'accesso ha avuto successo, consentendo l'apertura del rispettivo pannello.

- **public void effettuaLogout():**

Questo metodo si occupa semplicemente di invalidare la sessione utente corrente, impostando l'attributo privato `utenteLoggato` al valore di riferimento `null`.

- **public Utente getUtenteLoggato():**

Metodo di consultazione che restituisce l'oggetto `Utente` attualmente associato alla sessione attiva dell'applicazione.

4.3.4 I Metodi di Scrittura Transazionale (PostgreSQL + Caching)

Questi metodi implementano il principio cardine dell'architettura BCE + DAO: eseguono prima la persistenza fisica su PostgreSQL tramite le interfacce d'accesso e successivamente allineano la cache in memoria RAM.

- **public boolean registraNuovoUtente(...):**

Consente l'inserimento di un nuovo studente o docente.

1. Invoca `utenteDAO.registraUtenteDB` passando i dati piatti (nome, cognome, e-mail, login, password, ruolo e anno corso). La matricola viene omessa poiché, a livello relazionale, la colonna di destinazione sfrutta una sequenza automatica impostata come valore di default.
2. Se l'operazione ha successo sul DB, richiama il metodo interno `sincronizzaConDatabase()` per forzare la ricarica dei dati. Questo passaggio è fondamentale affinché l'ID utente autogenerato da PostgreSQL venga letto e mappato correttamente nella nostra cache a runtime.
3. Restituisce `true` per confermare l'esito positivo alla GUI di registrazione.

- **public void inviaRichiestaSpostamento(...):**

Gestisce l'inoltro di un ticket di variazione d'orario da parte di un docente.

1. Chiama il metodo di scrittura del database `richiestaSpostamentoDAO.inviaRichiestaDB` passando l'ID numerico della lezione da spostare (ricavato tramite `lezione.getIdLezione()`) e i dati del nuovo orario proposto
2. Istanza un nuovo oggetto temporaneo di tipo `RichiestaSpostamento` e lo aggiunge alla lista in memoria `richieste` per l'aggiornamento grafico immediato delle interfacce.

- **public void valutaRichiesta(RichiestaSpostamento richiesta, StatoRichiesta nuovoStato):**

Gestisce la delibera del coordinatore sulle richieste pendenti.

1. Invoca `richiestaSpostamentoDAO.valutaRichiestaDB` per aggiornare fisicamente lo stato del ticket su PostgreSQL.

2. Se lo stato impostato è 'APPROVATA', recupera l'ID della lezione reale interessata ed esegue una UPDATE sul database tramite `lezioneDAO.aggiornaOrarioLezioneDB` per aggiornare la pianificazione del calendario relazionale.
3. Successivamente, allinea l'oggetto in memoria modificando le proprietà della lezione reale tramite i metodi di scrittura della classe `Lezione` (`setGiorno`, `setOraInizio`, `setOraFine`).
4. Infine, aggiorna lo stato dell'oggetto richiesta in memoria a runtime.

- **public boolean inserisciNuovaAula(String nome):**

Invia la richiesta di inserimento di una nuova aula a PostgreSQL chiamando `aulaDAO.inserisciAulaDB`. Se la transazione ha successo sul DB, forza l'allineamento chiamando `sincronizzaConDatabase()` in modo da aggiornare i menu a tendina della pianificazione lezioni con l'ID autogenerato.

- **public boolean inserisciNuovoInsegnamento(...):**

Calcola l'ID numerico del docente titolare (passando -1 se non è presente alcun docente, per non violare l'integrità referenziale della chiave esterna) e invoca `insegnamentoDAO.inserisciInsegnamento` per la sincronizzazione finale all'avvenuta scrittura.

- **public boolean inserisciNuovaLezione(...):**

Preleva l'ID del corso e dell'aula ospitante e invoca la scrittura sul DB tramite `lezioneDAO.inserisciLezione`. Termine, sincronizza i dati per caricare l'ID autogenerato da PostgreSQL all'interno dell'oggetto `Lezione` in memoria.

4.3.5 I Metodi di Consultazione della Cache (Query in Memoria)

Questi metodi estraggono le informazioni richieste dalla GUI direttamente dalle liste presenti in memoria volatile, rappresentando l'aspetto di cache del pattern BCE + DAO, riducendo a zero le query ripetitive sul database(garantendo la velocità massima).

- **public List<Lezione> getLezioniPerStudente(Studente studente):**

Si occupa di filtrare la lista completa delle lezioni confrontando l'anno di corso dell'insegnamento con l'anno di corso dello studente. Questa operazione di filtraggio avviene senza gravare sul DBMS con interrogazioni di selezione continue.

- **public List < Lezione > getLezioniPerDocente(Docente docente):**

si occupa di filtrare la lista delle lezioni in memoria estraendo solo quelle il cui insegnamento ha come docente titolare un ID utente corrispondente a quello del docente passato come parametro.

- **public List<RichiestaSpostamento> getRichiesteInAttesa():**

Filtra la lista delle richieste in memoria estraendo esclusivamente i ticket che si trovano ancora in stato di valutazione ('IN_ATTESA'), per consentirne l'approvazione da parte del coordinatore.

- **getAule() / getInsegnamenti() / getDocenti():**

Metodi getter che restituiscono i riferimenti alle collezioni in memoria, utilizzati dalle interfacce di boundary per popolare dinamicamente i menu a tendina della pianificazione.

4.4 Tabella Riassuntiva dello Strato di Controllo

Al fine di fornire un quadro operativo e strutturato dello strato di controllo, viene sintetizzato nella Tabella 2 tutti i metodi implementati nella classe `Controller`, specificando la funzione di ciascuno

Metodo	Funzione e Descrizione Operativa
<code>getInstance()</code>	Inizializza ed estrae l'unica istanza globale del Controller (Pattern Singleton)
<code>effettuaLogin()</code>	Verifica le credenziali sul database ed istanzia a runtime l'oggetto del Model corretto.
<code>effettuaLogout()</code>	Invalida la sessione utente impostando il riferimento dell'utente loggato a <code>null</code> .
<code>getUtenteLoggato()</code>	Restituisce il riferimento all'oggetto dell'utente attualmente autenticato.
<code>getLezioniPerStudente()</code>	Filtra in tempo reale all'interno della cache l'orario delle lezioni per l'anno dello studente.
<code>getLezioniPerDocente()</code>	Filtra in tempo reale all'interno della cache l'orario per il docente titolare.
<code>getRichiesteInAttesa()</code>	Filtra in tempo reale l'elenco dei ticket di spostamento orario in stato 'IN_ATTESA'.
<code>inviaRichiestaSpostamento()</code>	Registra la proposta di spostamento orario a database e aggiorna la cache in memoria.
<code>valutaRichiesta()</code>	Aggiorna lo stato del ticket e, se approvato, modifica l'orario della lezione sia su DB che nel Model.
<code>sincronizzaConDatabase()</code>	Svuota la cache e ricostruisce in memoria l'intero grafo degli oggetti leggendo a cascata PostgreSQL.
<code>registraNuovoUtente()</code>	Registra in modo transazionale un utente (Joined Table) su DB ed esegue il re-sync della cache.
<code>inserisciNuovaAula()</code>	Inserisce un'aula fisica su PostgreSQL ed esegue la sincronizzazione per aggiornare lo stato.
<code>inserisciNuovoInsegnamento()</code>	Registra una materia su DB associandole i CFU e il docente titolare, ed esegue il re-sync.
<code>inserisciNuovaLezione()</code>	Pianifica una lezione memorizzando gli ID di corso e aula su DB, ed esegue il re-sync.
<code>getAule() / getInsegnamenti()</code>	Restituiscono i riferimenti alle liste in memoria per popolare i menu a tendina della GUI.

Tabella 2: Quadro riassuntivo dei metodi dello strato di Controllo (Controller).

5 Lo Strato di Persistenza dei Dati: Il Pattern DAO

Per implementare una persistenza dei dati robusta, scalabile e slegata dalle specifiche del modello grafico o logico, si è integrato nel sistema il pattern **Data Access Object (DAO)**. Questo strato si occupa di separare le specifiche astratte delle operazioni sui dati (le interfacce) dalla loro implementazione fisica mediante query SQL ed il protocollo JDBC per PostgreSQL.

5.1 La Gestione della Connessione: Il Package `database_connection`

La connessione fisica al database PostgreSQL è interamente gestita dalla classe di utilità `ConnessioneDatabase` posizionata nel package `database_connection`. Ho progettato questa classe applicando il pattern creazionale **Singleton**, il quale prevede che si acceda al database attraverso un'unica connessione senza che se ne creino diverse, le quali potrebbero generare una situazione dove i dati vengono modificati simultaneamente, e per assicurare l'esistenza di un'unica connessione attiva durante tutta l'esecuzione del programma.

La classe opera secondo le seguenti specifiche tecniche:

- **Caricamento dinamico (JDBC):** All'atto dell'inizializzazione del Singleton, il costruttore privato carica dinamicamente a runtime il driver di PostgreSQL (`org.postgresql.Driver`) mediante l'istruzione `Class.forName()`. Questa scelta garantisce una notevole flessibilità qualora in futuro si decidesse di cambiare il DBMS di riferimento.
- **Gestione della sessione:** La connessione fisica viene stabilita richiamando il metodo `DriverManager.getConnection()`.
- **Metodo di accesso globale:** Il metodo statico `getInstance()` controlla qualora l'istanza privata della classe sia nulla o se la connessione sottostante sia stata chiusa dal server. In ambo i casi, provvede a istanziare un nuovo oggetto, garantendo che le classi d'accesso ricevano sempre una connessione valida.

5.1.1 Gestione delle Risorse JDBC e Sicurezza nelle Query

L'implementazione concreta dello strato DAO, posizionata nel package `implementazioneDao`, segue alcune buone pratiche fondamentali per garantire la stabilità delle connessioni al database PostgreSQL e la sicurezza dei dati inseriti dagli utenti. **Uso del costrutto Try-With-Resources:** Per evitare che le connessioni rimangano aperte per errore in memoria (il problema del *resource leaking*), l'apertura degli oggetti JDBC come `Connection`, `PreparedStatement` e `ResultSet` viene gestita direttamente nella clausola del `try`. Con questa tecnica, introdotta da Java 7, la chiusura e il rilascio delle risorse verso il database avvengono in modo automatico appena termina il blocco di codice. Questo succede sia in caso di successo sia se dovesse essere sollevata una `SQLException`, eliminando la necessità di scrivere i vecchi e pesanti blocchi `finally` annidati. **Uso dei `PreparedStatement`:** Per l'esecuzione delle query non viene mai usata la concatenazione diretta di stringhe con gli input provenienti dai form. L'utilizzo sistematico dei `PreparedStatement` permette di definire la query inserendo dei punti interrogativi (?) come segnaposto. Questa scelta consente al database di precompilare la query e riutilizzarla (migliorando le prestazioni), ma soprattutto azzera il rischio di attacchi di tipo *SQL Injection*, poiché i caratteri speciali inseriti maliziosamente nei campi di testo vengono trattati dal sistema come semplice testo e mai come codice SQL eseguibile.

5.2 Le Interfacce d'Accesso: Il Package `dao`

Nello scrivere le interfacce nel package `dao`, è seguita una regola fondamentale per mantenere la totale indipendenza dello strato di persistenza(database) rispetto al modello di dominio: nessun

metodo dichiarato nelle interfacce DAO accetta o restituisce oggetti del Model,garantendo ,dunque, la separazione dei ruoli,garantendo la coerenza dei dati.

I metodi lavorano unicamente con tipi primitivi (`int`, `boolean`), classi standard di Java (`String`) o liste di appoggio (`ArrayList<String>`). Ad esempio, per registrare un utente o una lezione, non passo l'oggetto al DAO, ma invio i singoli attributi grezzi.In modo analogo,per leggere i dati dal database, vengono passati ai metodi del DAO delle liste di appoggio vuote le quali saranno riempite dalle classe di implementazione durante la scansione delle righe del database.Sarà poi compito del *Controller* estrarre i dati dalle liste e istanziare gli oggetti del Model.

Ogni metodo dichiara la clausola `throws SQLException`, demandando la cattura dell'eccezione e la conseguente notifica d'errore all'interfaccia utente direttamente alla classe di controllo.

5.3 L'Implementazione PostgreSQL: Il Package implementazioneDao

Le classi contenute nel package `implementazioneDao` realizzano concretamente le interfacce d'accesso, traducendo le chiamate in istruzioni SQL eseguite su PostgreSQL. Nella scrittura di queste classi ho applicato principi di sicurezza e stabilità del codice:

- **Prevenzione di SQL Injection:** Non sono mai stati concatenati i parametri direttamente all'interno delle stringhe delle query (soluzione altamente vulnerabile agli attacchi di iniezione di codice e sensibile agli errori di formattazione delle date o delle stringhe con apici)Al contrario, vengono istanziati esclusivamente oggetti di tipo `PreparedStatement`, in cui i parametri dinamici vengono inseriti in modo sicuro tramite i segnaposto `?` e impostati con i metodi di tipo `setString()`, `setInt()` o `setTime()`,al fine di garantire la massima sicurezza.
- **Uso del Try-with-resources:** Per garantire la chiusura automatica delle risorse anche in caso di eccezioni a runtime, ho implementato ogni query con la sintassi del *try-with-resources*. Questa pratica elimina la necessità di scrivere blocchi `finally` complessi e ridondanti, garantendo la pulizia del codice valutata da analizzatori automatici.

5.4 Tabella Riassuntiva delle Operazioni del DAO

Per documentare l'organizzazione, ho riassunto nella Tabella 3 la mappatura tra le interfacce d'accesso,le classi fisiche di implementazione PostgreSQL e le specifiche query SQL eseguite sul database relazionale.

Interfaccia DAO	Classe Implementazione	Operazioni SQL Mappate (Query)
UtenteDAO	UtenteImplementazionePostgresDAO	- INSERT transazionale su "Utente" e "Studente" / "Docente". - SELECT per convalida credenziali con controllo dello stato di abilitazione. - SELECT con JOIN per anagrafica utente loggato e lista docenti.
AulaDAO	AulaImplementazionePostgresDAO	- INSERT di una nuova aula fisica. - SELECT per l'estrazione dell'elenco completo delle aule.
InsegnamentoDAO	InsegnamentoImplementazionePostgresDAO	- INSERT di un corso associato alla FK del docente titolare. - SELECT per estrarre tutti i corsi attivi del piano di studi.
LezioneDAO	LezioneImplementazionePostgresDAO	- INSERT di una lezione legata alle FK di aula e insegnamento. - SELECT per estrarre l'intero orario programmato. - UPDATE di orario e giorno a seguito di spostamento.
RichiestaSpostamentoDAO	RichiestaSpostamentoImplementazioneDAO	- INSERT di una proposta di riprogrammazione di una lezione. - UPDATE dello stato del ticket (approvato/rifiutato). - SELECT di tutte le richieste inserite a sistema.

Tabella 3: Quadro delle interfacce d'accesso e delle operazioni SQL gestite dal DAO.

5.5 Dettaglio dell'Implementazione Fisica dei Data Access Objects

In questa sezione viene analizzata la logica operativa e le query SQL che ho implementato all'interno delle singole classi del package `implementazioneDao`, evidenziando come ciascuna di esse traduca le specifiche astratte delle interfacce in transazioni concrete su PostgreSQL.

5.5.1 La classe `UtenteImplementazionePostgresDAO`

Questa classe si fa carico della persistenza della gerarchia degli utenti, strutturata a database tramite la strategia del *Joined Table*. Ho dovuto affrontare due problematiche principali:

- Quando il sistema registra un nuovo utente, il metodo `registraUtenteDB` esegue una prima `INSERT` sulla tabella padre `Utente`. Poiché l'ID della chiave primaria è un intero autogenerato dal DBMS (`SERIAL`), ho sfruttato la clausola `SQL RETURNING "ID_Utente"` per recuperare al volo l'ID generato all'atto della scrittura. Subito dopo, all'interno dello stesso blocco transazionale, il metodo esegue una seconda `INSERT` sulla tabella figlia (`Studente` o `Docente`), referenziando l'ID appena recuperato come chiave esterna. Questo garantisce l'atomicità dell'inserimento.
- Per evitare di sovraccaricare il database con query multiple di verifica, ho progettato il metodo `loginDB` affinché esegua una singola query SQL ottimizzata con due `LEFT JOIN` simultanee verso le tabelle figlie:

```
SELECT u.ID_Utente, d.IsResponsabile, s.ID_Utente AS IsStudente ...
```

Verificando la presenza di valori non nulli nei campi restituiti dalle join, il metodo è in grado di dedurre istantaneamente se l'utente sia uno `STUDENTE`, un `DOCENTE` o un `RESPONSABILE` in un solo viaggio verso il dbms.

Breve nota sulla base di dati: L'utilizzo della clausola `RETURNING` Nel contesto della persistenza dei dati, la gestione dell'ereditarietà dove la superclasse e le sue sotto-classi sono mappate su tabelle relazionali distinte, introduce una problematica legata alla sincronizzazione delle chiavi primarie. All'interno del metodo `registraUtenteDB` della classe `UtenteImplementazionePostgresDAO`, la registrazione di un nuovo account richiede un inserimento multiplo: prima i dati anagrafici e le credenziali nella tabella genitore `Utente`, e successivamente le informazioni di ruolo nella tabella figlia corrispondente (`Studente` o `Docente`).

Per associare correttamente il record figlio al record padre, il sistema deve conoscere la chiave primaria auto-generata (l'`ID_Utente`) nell'esatto istante del primo inserimento. Al fine di risolvere questo problema in modo robusto, si è scelto di implementare la clausola `RETURNING` offerta nativamente dal DBMS PostgreSQL. Come visibile nella definizione della query:

```
INSERT INTO public."Utente" ("Nome", "Cognome", "Email", "Login", "Password")  
VALUES (?, ?, ?, ?, ?) RETURNING "ID_Utente";
```

questa istruzione ordina al database di effettuare l'inserimento e di restituire contestualmente il valore appena generato per la chiave primaria, catturabile direttamente tramite il `ResultSet` di Java.

Questa scelta progettuale si discosta da approcci più semplici, come l'esecuzione di una query successiva del tipo `SELECT MAX("ID_Utente")`. Tali soluzioni, infatti, espongono il sistema a gravi rischi di concorrenza, qualora due utenti effettuassero la registrazione nello stesso esatto millisecondo, il sistema potrebbe assegnare a un utente l'ID appartenente all'altro, corrompendo l'integrità referenziale del database. L'utilizzo di `RETURNING`, garantisce assoluta *thread-safety* e ottimizza le prestazioni riducendo le interazioni.

5.5.2 La classe `AulaImplementazionePostgresDAO`

Questa classe implementa le interrogazioni per l'entità *Aula*. Il metodo `leggiAuleDB` esegue una query di selezione completa (`SELECT *`) ed estrae i record popolandoli in modo iterativo le liste di appoggio passate dal *Controller*. Non essendoci vincoli complessi o chiavi esterne in questa tabella, l'operazione è altamente lineare e veloce.

5.5.3 La classe `InsegnamentoImplementazionePostgresDAO`

Gestisce la memorizzazione dei corsi accademici. Nel metodo `inserisciInsegnamentoDB`, si è dovuta gestire l'integrità referenziale della chiave esterna verso la tabella `Docente`. Nel caso in cui un corso venga inserito senza un docente titolare assegnato, il metodo intercetta il valore sentinella `-1` e provvede a invocare il metodo JDBC `ps.setNull(4, Types.INTEGER)`, memorizzando un valore nullo coerente sul database ed evitando violazioni di vincolo di integrità fisica.

5.5.4 La classe `LezioneImplementazionePostgresDAO`

Questa classe gestisce la pianificazione dell'orario settimanale e presenta due aspetti tecnici rilevanti:

- **Mappatura dei Tipi Temporal:** Il database PostgreSQL memorizza gli orari di inizio e fine lezione tramite il tipo nativo `TIME`. In fase di lettura, il metodo `leggiLezioniDB` recupera questi valori convertendoli in oggetti di tipo `java.sql.Time` e, successivamente, li traduce in stringhe formattate (escludendo i secondi) per consentire un facile parsing a runtime da parte della classe `LocalTime` del Model.
- **Aggiornamento dell'Orario (Riprogrammazione):** Il metodo `aggiornaOrarioLezioneDB` esegue una query di tipo `UPDATE` sulla tabella `Lezione` modificando i campi `Giorno`, `OraInizio` e `OraFine` in corrispondenza dell'ID della lezione, consolidando fisicamente sul DB le decisioni di spostamento prese dal coordinatore.

5.5.5 La classe `RichiestaSpostamentoImplementazioneDAO`

Gestisce la persistenza dei ticket di spostamento d'orario. Il metodo `valutaRichiestaDB` esegue un'operazione di `UPDATE` mirata sulla colonna `Stato` della tabella `RichiestaSpostamento`, modificando lo stato da `'IN_ATTESA'` ad `'APPROVATA'` o `'RIFIUTATA'` sulla base della chiave primaria del ticket. Anche in questa classe, l'uso di query parametrizzate garantisce la stabilità del sistema contro tentativi di manomissione degli input.

6 Sull'utilizzo delle Mappe

Viene riconosciuto che le funzionalità della presente applicazione sarebbero potute essere ampliate verso svariate direzioni che avrebbero garantito maggiore sicurezza per gli utenti, maggiore velocità d'utilizzo o maggiori funzionalità; si sarebbe potuto espandere il progetto andando a disegnare un'interfaccia grafica più pulita e accattivante, oppure implementare un sistema di crittografia al fine di potenziare la sicurezza del sistema e garantire la privacy degli utenti. Tuttavia ho preferito approfondire qualcosa che sia affine a me, ai miei interessi e alle mie attitudini: ho dunque scelto di ottimizzare il sistema, nell'ambito dell'interrogazione al database, nei metodi già esaminati nel dettaglio precedentemente in questo medesimo documento, andando ad ottimizzare la maniera in cui si accede ai dati caricati in memoria volatile dal database mediante una particolare struttura dati, quella delle mappe, che può migliorare le prestazioni del sistema, anche se

nella demo presentata questo miglioramento non sarà affatto percepibile, in quanto la mole di dati presente nel sistema non ci permette di apprezzare queste differenze.

6.1 Cosa sono le mappe

Una **HashMap** è una struttura dati avanzata basata sul concetto di dizionario, che memorizza le informazioni sotto forma di coppie *Chiave-Valore*. A livello teorico, essa utilizza una funzione matematica (funzione di hash) per convertire la chiave univoca nell'indice di un array interno, permettendo così di memorizzare, cercare e recuperare i dati con una complessità temporale costante pari a $\mathcal{O}(1)$.

6.2 Uso delle mappe

Durante la fase di sincronizzazione, quando andiamo a ricostruire i collegamenti tra gli oggetti (ad esempio assegnare l'aula corretta a una determinata lezione), si è deciso di usare delle strutture dati di tipo **Map** (nello specifico **HashMap**) invece delle classiche **List**.

Il motivo principale di questa scelta architetturale è l'ottimizzazione massima della velocità di ricerca. Se avessimo salvato le aule estratte in una lista normale, per trovare l'aula associata alla chiave esterna restituita dal database avremmo dovuto fare un ciclo **for** e scorrere ogni singola aula una per una (complessità $\mathcal{O}(n)$). Nel peggiore dei casi, se l'aula cercata si fosse trovata alla fine della lista di grandezza n , il sistema avrebbe usato tempo n e n risorse computazionali per fare i controlli.

Usando una Mappa, invece, andiamo ad associare direttamente la chiave univoca (l'ID numerico del database) al suo valore (l'oggetto Java reale). La **HashMap** usa un meccanismo matematico chiamato *hashing* che calcola istantaneamente la cella di memoria dove si trova l'oggetto partendo semplicemente dal suo ID. In questo modo, usando l'istruzione `mappaAule.get(id)`, il programma fa un salto diretto e recupera l'oggetto al volo. Questo ci garantisce un tempo di accesso costante (complessità $\mathcal{O}(1)$) indipendentemente dalla mole dei dati, rendendo il caricamento iniziale nettamente più fluido.

6.2.1 Dai Cicli Nidificati all'uso delle HashMap

Durante la fase di sincronizzazione, eseguita all'avvio nel file **Controller.java** tramite il metodo `sincronizzaConDatabase()`, il sistema deve affrontare il classico problema dell'Object-Relational Mapping (ORM): tradurre le tabelle piatte del database relazionale (basate su chiavi esterne numeriche) in un vero e proprio grafo di oggetti Java (basato su riferimenti di memoria).

Il Problema e la Soluzione Iniziale Prendiamo come esempio l'istanziatura di un **Insegnamento**, il quale richiede al suo costruttore il riferimento all'oggetto **Docente** titolare. Dal database, leggendo la tabella *Insegnamento*, otteniamo solo un numero intero (la Foreign Key **FK_Docente**).

In una prima fase di analisi, la soluzione più intuitiva per recuperare l'oggetto **Docente** corretto consisteva nell'utilizzare le classiche **List**:

1. Salvare tutti i docenti estratti in una `List<Docente>`.
2. Durante la lettura degli insegnamenti, per ogni riga estratta, scorrere l'intera lista dei docenti con un ciclo **for** fino a trovare l'oggetto avente l'ID coincidente con la Foreign Key.

Questa soluzione "ingenua", basata su cicli nidificati, presentava però un grave difetto di scalabilità. Se il sistema avesse N docenti e M insegnamenti, la complessità computazionale dell'algoritmo sarebbe stata $\mathcal{O}(N \times M)$. Al crescere dei dati (specialmente con l'aggiunta delle

Lezioni, che dipendono da aule e insegnamenti), il tempo di caricamento sarebbe cresciuto in modo quadratico, causando lentezza.

la Soluzione Ottimizzata Per risolvere il problema, ho riconsiderato la natura dei dati: le chiavi primarie del database (ID) sono valori univoci. La struttura dati ideale per recuperare un valore in base a una chiave univoca in Java non è la Lista, ma la Mappa (nello specifico, l'implementazione `HashMap`)